

Bar-Ilan University

אוניברסיטת בר-אילן

M.Sc. Research Proposal

Learning Unknown Games from Interaction:
Diagnosing and Fixing Small Open Models

למידת משחקים לא מוכרים מתוך אינטראקציה:
אבחון ותיקון של מודלי שפה פתוחים קטנים

Student:
Adam Celermajer
אדם סלרמאייר [TODO: confirm]

Supervisor:
[**SUPERVISOR – TODO**]

Department of Computer Science and Artificial Intelligence
המחלקה למדעי המחשב ובינה מלאכותית
Bar-Ilan University, Ramat Gan, Israel

October 2026

Contents

1 Introduction **1**

2 Why Learning a Game from Interaction Is Hard **2**

3 Related Work **2**

4 Work So Far and Initial Results **3**

4.1 The environment (January–April 2026) 3

4.2 A rule-learning agent (January–July 2026) 3

4.3 The goal-recognition study (May–June 2026) 4

4.4 Direct player and AVO (July–September 2026) 4

4.5 The Qwen 3.8-27B thread (September–October 2026) 5

4.6 What we have learned so far 6

5 Research Goal, Hypotheses and Plan **6**

5.1 Goal and design 6

5.2 Hypotheses 7

5.3 Work plan 7

References **9**

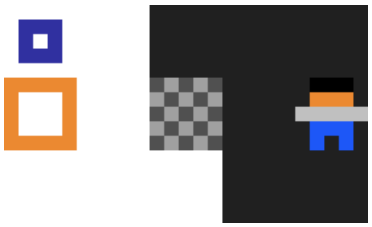
1 Introduction

Imagine an agent dropped into a game it has never seen. It receives a small grid of numbers and a picture. It is told which buttons exist (`ACTION1`, `ACTION2`, …) but not what they do. Nobody tells it the rules. Nobody tells it what winning means. It must find out by pressing buttons and watching what changes.

This is the setting of ARC-AGI-3 (ARC Prize Foundation, 2026), and it is a clean test of skill acquisition (Chollet, 2019): an agent that succeeds must form, test and revise hypotheses rather than recall answers.

PuzzleScript games. We study this setting with PuzzleScript (Lavelle, 2013), a small language for turn-based grid puzzles. A PuzzleScript game has four parts: *sprites* on a grid (player, walls, crates, targets), *rewrite rules* that say how the grid changes after an input ("a player moving into a crate pushes it"), a *win condition* ("every target has a crate"), and an ordered list of *levels* that reuse the same rules on harder layouts. Our repository contains 206 such games. Some are close to classic Sokoban. Others add their own twist: in *IceCrates* the player slides until it hits something, and crates pushed into water fill it; in *Knightoban* some crates move like chess knights, and a separate key commits the move.

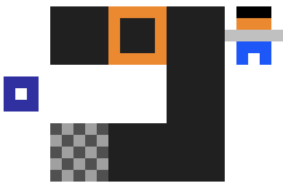
Running example: Modality. Figure 1 shows the first level of *Modality* as the agent sees it. The hidden rules are short. The player cannot step directly between white and black floor; the checkered tile bridges them. A crate can be pushed only when the player and the crate stand on compatible terrain. One button (`ACTION7`) is undo. The level is won when every crate is on a target. None of this is given to the agent. We use Modality throughout to make each difficulty concrete.



(a) Rendered frame, level 1

```
0 1 2 2 2
3 1 4 2 5
1 1 1 2 2
```

(b) Grid the agent receives



(c) Level 2

Figure 1: Modality. In (b) each number is an anonymous texture ID: 0 target, 1 white floor, 2 black floor, 3 crate, 4 checkered bridge, 5 player on black floor. The agent is not told these meanings, and the player's ID changes with the floor it stands on. Solving level 1 requires crossing to the white side through the bridge and pushing the crate up onto the target.

What this proposal covers. Since January 2026 we have built an ARC-AGI-3–compatible PuzzleScript environment, a rule-learning agent, a goal-recognition study with human participants, a direct game-playing agent, and an adaptation of NVIDIA's Agentic Variation Operators (AVO) (Chen et al., 2026). With these tools, frontier models and some *small* recent models recover hidden mechanics and solve whole games. A capable open model, Qwen 3.8-27B, does not. The proposed thesis asks why, along three axes—the harness, model scale, and pretraining exposure—and aims to build a harness under which the small open model succeeds.

2 Why Learning a Game from Interaction Is Hard

Solving an unknown game is not one skill. Table 1 decomposes it into seven parts. Each row gives a definition and a real example from our runs. This decomposition is the backbone of the proposal: the failure taxonomy in Section 5 labels each failed run with one of these categories, plus an eighth category, *protocol*, for failures of the agent's interface rather than of the game.

Challenge	What makes it hard	Example from our runs
Perception	IDs are anonymous; identical pictures share an ID; hidden objects are not shown.	In Modality the player's ID is 5, 6 or 7 depending on the floor below it; the agent must realise these are one object.
Action semantics	Buttons are unlabeled; some are not movements.	On Modality and Knightoban, Qwen 3.8-27B never identified `ACTION7` as undo. It called it "fires a shot", a "jump opposite the facing direction", and "contextual movement".
Rule induction	Effects depend on context.	Pushing in Modality depends on terrain. A GPT-5.6 Luna agent isolated this with controlled tests from one saved state; Qwen 3.8-27B instead built a new "wall map" for each level.
Goal recognition	The win condition is never stated.	Qwen 3.8-27B took the crate for "a goal frame you enter from below", and in Knightoban guessed "collect all red squares".
Planning	Moves can be irreversible; crates deadlock in corners.	A Qwen 3.8-27B run pushed the crate into a corner and concluded it was "permanently stuck". Undo would have recovered it.
Transfer	Later levels reuse rules on new layouts and add new mechanics.	A Qwen 3.8-27B note generalised from level 1: "unreachable object = decoy, reachable object = goal"; this rule misled it on level 2.
Credit assignment	A win may happen by accident.	Qwen 3.8-27B won Modality level 1 while "testing if the frame is enterable", and the win reinforced its wrong goal theory.

Table 1: The challenge decomposition, with examples taken from saved runs (Section 4).

These parts interact. A misread action (undo as jump) makes a reversible mistake look irreversible, which turns a rule-induction error into a planning dead end. A wrong goal makes every later plan wrong, even when the rules are right. Measuring the parts separately is what lets us say *where* an agent fails.

3 Related Work

Interactive reasoning benchmarks. ARC-AGI-3 evaluates agents on interactive grid games with no instructions (ARC Prize Foundation, 2026). BALROG (Paglieri et al., 2025) and AgentBench (Liu et al., 2024) show that long-horizon game play remains far below static-benchmark performance. WorldTest evaluates learned world models through environment queries rather than reward (Warrier et al., 2026). PuzzleJAX compiles PuzzleScript into fast environments (Earle et al., 2025); we instead keep the original engine and expose it through the ARC-AGI-3 interface, so the same agent code runs on both.

Harnesses. The code around a model matters. ReAct interleaves reasoning and tool calls (Yao et al., 2023). SWE-agent showed that interface design alone changes success rates (Yang et al., 2024). WorldCoder learns an executable world model by writing code (Tang et al., 2024). AVO

turns a coding agent into an evolutionary variation operator (Chen et al., 2026), and NVIDIA reports that it reaches the full score on the public ARC-AGI-3 set with a frontier model (NVIDIA, 2026). RSIAgent lets open models outperform closed ones through autonomous exploration and frozen memory, without training (Zhu et al., 2026). Small format choices also matter: prompt formatting can swing accuracy by tens of points (Sclar et al., 2024), and forced output formats can hurt reasoning (Tam et al., 2024).

Priors. Humans learn new video games fast because of prior knowledge about objects and game conventions; hiding these cues slows them sharply (Dubey et al., 2018). Agents that explicitly propose and revise causal theories approach human efficiency in Atari-like games (Tsividis et al., 2021). This suggests two failure sources we test separately: missing priors, and failure to revise.

Scale and training data. Loss scales smoothly with size (Kaplan et al., 2020), but abilities can appear abruptly, sometimes as a metric artifact (Schaeffer et al., 2023). Exposure to similar data inflates measured ability (Magar and Schwartz, 2022); it can be detected indirectly (Golchin and Surdeanu, 2024) or, for open corpora, searched directly (Elazar et al., 2024). Qwen's training data is closed (Yang et al., 2025), so we rely on indirect probes and optional fine-tuning (Hu et al., 2022).

4 Work So Far and Initial Results

This section follows the work in time order. Each subsection states what was built and what it taught us.

4.1 The environment (January–April 2026)

The PuzzleScript engine is an npm package. A Node.js service runs the games, and a Python FastAPI service exposes them through the public ARC-AGI-3 REST contract (reset, actions, scorecards). Any ARC-AGI-3 agent can therefore play our games without changes. Observations use `visible-cell-v1`: one integer per rendered tile, as in Figure 1(b), plus a lossless PNG. Game names, sprite names and action meanings are hidden.

On top of this we built a benchmark layer (catalog sampling, scorecards for solving, goal and next-frame claims, and a live monitor) and a web workbench for launching and replaying runs. Search produced replay-verified winning paths for the first playable level of 30 sampled games, and a whole-game reference for Sokoban (49 actions over two levels). These references are used only for evaluation, never shown to agents.

4.2 A rule-learning agent (January–July 2026)

Our first agent tried to make learning explicit. After each transition, an LLM proposed a natural-language rule ("`ACTION1` moves the player up into empty floor"). The rule was compiled into a logical rule over relative cells, checked against the full transition timeline, and stored only if no recorded transition contradicted it. A planner then searched with the verified rules, and the LLM proposed subgoals only when no rule-based plan existed. In the recorded runs the agent was given the object legend and the instruction to complete the level.

In short runs on Sokoban the agent learned correct, verified movement, wall and push rules. It did not complete a level in the recorded runs. The lesson we drew was that a hand-designed rule language is too narrow for the variety of PuzzleScript mechanics, and that many LLM calls

per step make it slow. We therefore moved to general coding agents that write their own world models [TODO: confirm this summary of the rule-agent outcome].

4.3 The goal-recognition study (May–June 2026)

Goal recognition is the least-studied part of Table 1, so we measured it on its own. We curated 50 PuzzleScript games and asked four frontier models (Claude Opus 4.8, GPT-5.5, Kimi K2.6, DeepSeek V4 Pro) to state the goal and win condition of each game. Two factors were crossed: the *evidence* (the first frame only, or the first frame plus three random actions) and the *input* (text only, or text plus the first image; DeepSeek received text only). This gave 700 model predictions. In parallel, a web study collected 114 free-text goal descriptions from 38 human participants, covering 48 of the 50 games [TODO: describe what humans saw].

Evidence	Text only	Text + image	Predictions
First frame	4 models	3 models	350
Frame + 3 random actions	4 models	3 models	350
Human study	38 participants, 48 games		114 answers

Table 2: Goal-recognition data collected (50 games). Correctness labels (correct / partial / wrong) have not yet been assigned.

Correctness has not been scored yet. A review tool lets an annotator play each game and label a prediction correct, partial or wrong; no labels have been recorded so far, so we report no accuracies. Two observations already shape the plan. First, the conditions are confounded: first-frame text prompts used the game's own level glyphs (`P`, `*`, `O`, …), while trajectory prompts used anonymous IDs. The scoring stage must separate evidence from representation. Second, glyphs invoke priors, for better or worse. From the Sokoban glyphs, Claude Opus 4.8 answered "Push the box (*) onto the target/goal (O), a Sokoban-style puzzle", but it read `@` as the player, as in the common Sokoban text format; in this game `@` means a crate on a target. From anonymous IDs after three actions it said only "Push or move the player token (2) toward a target/goal location".

4.4 Direct player and AVO (July–September 2026)

Direct player. One persistent model session receives each frame and returns exactly one action. The *baseline+* condition adds three note-taking tools and requires a `strategy.md` file. A harness ladder can run the same condition through the Codex SDK, native function tools, or plain text.

AVO. In our from-scratch adaptation of AVO (Chen et al., 2026), an *inner player* researches the game (with checkpoints and at most two solver self-checks), an *outer optimizer* evolves a lineage of strategies without touching the game, and a *fresh evaluator* scores a candidate only by the levels its frozen `solver.py` completes alone in a new game, with no model calls. All sessions run in a Docker sandbox without network or game sources.

Results. Table 3 lists the best fresh-solver scores. A post-hoc audit against the game sources found that the core mechanic of each solved game was recovered from observation, including corrected false hypotheses (e.g., the two-crate "train" collision in IceCrates). DeepSeek V4.1 Flash, a small recent model, matched the frontier models on all three games. Separately, GPT-5.6 Luna fully solved three of six further games in both repeats (Feeling Like Filling 6/6, Mowing 4/4, Fatigued Square Maze 8/8).

Game (levels)	Player / optimizer model (effort)	Best solver	Actions
Modality (4)	GPT-5.6 Luna (high) / GPT-6 Astra (low)	4/4	263
Knightoban (6)	GPT-5.6 Luna (high) / GPT-6 Astra (low)	6/6	255
IceCrates (6)	GPT-6 Astra (low) / GPT-6 Astra (high)	6/6	138
Modality (4)	DeepSeek V4.1 Flash (max) / same	4/4	263
Knightoban (6)	DeepSeek V4.1 Flash (max) / same	6/6	216
IceCrates (6)	DeepSeek V4.1 Flash (max) / same	6/6	90
Enqueue (14)	GPT-6 Luna (high) / GPT-6 Sol (high)	8/14	–
Enqueue (14)	DeepSeek V4.1 Flash (medium) / same (max)	8/14	–

Table 3: Best fresh-solver score per AVO campaign; "Actions" is the length of the winning fresh evaluation.

4.5 The Qwen 3.8-27B thread (September–October 2026)

Qwen 3.8-27B is open and cheap, which makes it the natural model to study in depth. It fails where small closed models succeed. Table 4 lists every run in which it played. Ten further launches failed before the model acted (Docker down, an SDK option error, isolation guards); these are infrastructure failures and are excluded.

Provider	Harness	Effort	Game	Actions	Levels	How it ended
unpinned	AVO research	medium	Modality	228	1/4	stopped by hand
unpinned	baseline+	max	Modality	124	1/4	stopped by hand
unpinned	baseline+	high	Modality	36+53	1/4	parse error, continued, stopped
unpinned	baseline+	high	IceCrates	2	0/6	parse error
Alibaba	baseline+	high	Modality	100	0/4	action budget
Alibaba	baseline+	high	Knightoban	120	0/6	action budget
Alibaba	baseline+	high	IceCrates	100	3/6	in progress [TODO: final]

Table 4: All runs in which Qwen 3.8-27B played. "Stopped by hand" means a run that was clearly not progressing was stopped to save budget. The Alibaba runs had budgets of 100 (Modality) and 120 actions; the IceCrates run was still in progress at the time of writing.

Harness problems we found and fixed. (1) With an enforced JSON action schema, the model made 0/2 note-tool calls and wrote the note inside its answer instead; without the schema it made 2/2 calls. One harness choice silently disabled a tool. (2) Twice, the model wrote a sentence before a valid JSON action ("I've been stuck in a note loop…"). The strict parser ended the run; the parser now accepts such replies and asks again if a reply is unusable. (3) Provider control. OpenRouter routes an open model to any provider unless pinned. Our first provider choice, DeepInfra, accepts at most 30 images per request, so our long early runs could not have been served by it and ran on an unknown, possibly quantised provider. All new runs are pinned to Alibaba, the model's first-party provider, with fallbacks disabled.

What the model believed. We compared the notes of two unpinned 27B runs with a GPT-5.5 run in the same baseline+ harness, which solved Modality level 2 in 39 actions (3/4 levels overall).

The 27B model used the tools correctly, returned valid actions, and tracked coordinates and IDs. Its failure was in *forming and revising hypotheses*: it did not reach for common conventions (undo, Sokoban pushing), explained evidence away level by level, and credited an accidental win to the wrong cause. GPT-5.5 named the genre and the undo key early. The new pinned runs refine the picture. On Modality the 27B now called the crate a "pushable Sokoban box" and

Mechanic (truth)	Qwen 3.8-27B	GPT-5.5
`ACTION7` (undo)	"fires a shot"; "jumps one tile opposite the facing direction"	"likely reset/undo", then "undo one move"
Crate (pushable)	"goal frame entered from below"; "decoy"	"Sokoban-like pushable blocks"
Colour blocking	a new wall map per level	"light player cannot enter dark tiles directly"
Level-1 win	accidental; reinforced the wrong goal	"solved by pushing orange onto purple"

Table 5: Beliefs about Modality from the agents' own notes, compared with the game source (read only for this audit).

the target a "goal pad", but it still read `ACTION7` as a jump, pushed the crate into a corner, and never cleared level 1. On Knightoban it read the knight crates as collectibles and never found the win condition. On IceCrates it did identify undo, used it in its plans, and cleared three levels in its first 27 actions. These are few runs, not proof, but they point to action semantics, goal recognition and revision rather than protocol as the main remaining failures.

4.6 What we have learned so far

1. PuzzleScript games are a practical test bed in the style of ARC-AGI-3: 206 games, hidden rules and goals, and an engine that serves as ground truth.
2. A fixed, hand-designed rule language was too narrow. Agents that write their own code and solvers went much further.
3. Goal recognition can be measured on its own, but the input representation can carry priors (Sokoban glyphs), so it must be controlled.
4. Small recent models can learn and solve whole games inside a strong harness; see DeepSeek V4.1 Flash in Table 3.
5. For Qwen 3.8-27B, harness details (output schema, parser strictness, provider) silently changed outcomes. Once they were fixed, its failures concentrate in action semantics, goal recognition and hypothesis revision.

5 Research Goal, Hypotheses and Plan

5.1 Goal and design

Goal. Make Qwen 3.8-27B solve PuzzleScript games it currently fails, by first diagnosing why it fails. An agent is a model m with a harness h (prompt, tools, output format, memory, budgets). The outcome $L(m, h, g)$ is the number of levels completed in game g: by the agent's own play for the direct player, or by the frozen solver for AVO. We ask:

RQ1 (Where). Which challenge in Table 1, or the protocol, causes each failure of the 27B model?

RQ2 (Why). How much of the gap is explained by harness, by scale within the Qwen 3.8 family, and by training exposure?

RQ3 (Fix). Which harness changes raise L for the 27B model, and do they generalise to games not used to design them?

Following a correlational-then-interventional design, the work has two stages.

Stage A (correlational). (i) Run the 27B model under baseline+ and AVO on Modality, IceCrates and Knightoban, three repeats each, pinned to one provider. Label every failed

run with the taxonomy. (ii) Run a fixed-harness model sweep: the Qwen 3.8 family from `qwen3.8-flash` through 27B and the open 2.4T-A95B weights to Qwen 3.8 Max, plus small successful models (DeepSeek V4.1 Flash, GPT-6 Luna) and a large flagship released before ARC-AGI-3 [TODO: choose]. (iii) Probe familiarity without play: can the model write PuzzleScript that compiles in our engine, name a level's genre from its image, and predict the next grid of logged transitions?

Stage B (interventional). Change one harness component at a time, chosen from Stage A: revision scaffolds (list two competing explanations for each surprising transition, including "undo"; replay "what caused this win?" after each level); an explicit hypothesis ledger; a forced predict–act–compare loop; observation format (IDs, ASCII, frame diffs); code tools for search; budgets. Optionally, fine-tune an open Qwen model with LoRA on successful trajectories from *other* games.

Goal recognition as a condition. The goal-recognition study becomes a probe inside this design. The same game runs under three conditions: goal hidden (the normal task), goal *elicited* (the agent must state the goal before acting and revise it after each level), and goal *given* (the true win condition is in the prompt). If giving the goal fixes the 27B model, goal recognition is its bottleneck; if not, the problem lies in rules or planning. In parallel we score the 700 existing predictions and 114 human answers, which gives a model-vs-human baseline and separates evidence from representation.

Taxonomy and agreement. Each failed run gets one primary label: one of the seven challenges in Table 1, or protocol. An LLM labeller (Zheng et al., 2023) reads the run's notes, transitions and the true rules, and cites the step where the failure occurs. Two human annotators label at least 40 runs independently. We report Cohen's κ (Cohen, 1960) and use the LLM labels only if $\kappa \geq 0.6$ (Landis and Koch, 1977).

Measures. Levels solved (L, L/n_g, whole-game success); actions per level; protocol errors per turn; the distribution of failure labels; goal-statement correctness; tokens and cost. Provider, precision and dates are recorded for every run.

5.2 Hypotheses

H1 (Harness). Harness changes alone, with the model fixed, raise the 27B model's levels solved on held-out games. After protocol fixes, the most effective changes will target revision and action semantics, not format.

H2 (Scale). Within the Qwen 3.8 family, under one harness, levels solved rise with model size.

H3 (Pretraining). At similar size or cost, models that succeed also score higher on the familiarity probes, consistent with exposure to grid puzzles and their conventions.

H4 (Goal). Giving the 27B model the true goal improves its levels solved more than any single other intervention.

5.3 Work plan

The plan covers November 2026 to June 2027 (Table 6).

1. **Failure taxonomy.** 18 pinned 27B runs; labelling; κ. Score the goal-recognition predictions and human answers.
2. **Model sweep.** Qwen family and controls on the three games.
3. **Familiarity probes.** PuzzleScript writing, genre naming, next-frame prediction; optional LoRA fine-tuning.
4. **Harness interventions** (main contribution). Single-factor ablations, including the goal conditions; combine the best and evaluate on held-out catalog games.

5. **Trace comparison.** Successful vs. failed runs (Max vs. 27B, small closed models vs. 27B), coded with the taxonomy.

Task	Nov	Dec	Jan	Feb	Mar	Apr	May	Jun
1. Taxonomy, κ, goal-recognition scoring	•	•						
2. Model sweep		•	•					
3. Familiarity probes (optional fine-tuning)			•	•	•			
4. Harness interventions, held-out evaluation				•	•	•		
5. Trace comparison						•	•	
Thesis writing; paper submission							•	•

Table 6: Timeline (2026–2027).

Expected contributions. (1) A challenge decomposition and failure taxonomy for agents that learn games from interaction, with an LLM labeller validated against humans. (2) A scored model-vs-human comparison of goal recognition on PuzzleScript games. (3) A controlled decomposition of one open model's failure into harness, scale and training factors. (4) A harness, built from single-factor ablations, under which Qwen 3.8-27B solves games it currently fails, evaluated on held-out games.

Budget and risks. A 27B run costs about $2; a Qwen 3.8 Max run of similar length is estimated at about $20. The full plan needs a few hundred dollars of API cost [TODO: confirm budget source]; fine-tuning needs GPU access [TODO: confirm compute]. Hosted models can change, so we record provider and dates and treat the open-weight models as the primary subjects. To avoid fitting the harness to three games, final results use held-out games. Qwen's training data is closed, so the pretraining explanation can only be supported indirectly unless fine-tuning shows a causal effect; we will say so.

References

ARC Prize Foundation. ARC-AGI-3: A new challenge for frontier agentic intelligence. *arXiv preprint arXiv:2603.24621*, 2026. URL https://arxiv.org/abs/2603.24621.

Terry Chen, Zhifan Ye, Bing Xu, Zihao Ye, Timmy Liu, Ali Hassani, Tianqi Chen, Andrew Kerr, Haicheng Wu, Yang Xu, Yu-Jung Chen, Hanfeng Chen, Aditya Kane, Ronny Krashinsky, Ming-Yu Liu, Vinod Grover, Luis Ceze, Roger Bringmann, John Tran, Wei Liu, Fung Xie, Michael Lightstone, and Humphrey Shi. AVO: Agentic variation operators for autonomous evolutionary search. *arXiv preprint arXiv:2603.24517*, 2026. URL https://arxiv.org/abs/2603.24517.

François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019. URL https://arxiv.org/abs/1911.01547.

Jacob Cohen. A coefficient of agreement for nominal scales. *Educational and Psychological Measurement*, 20(1):37–46, 1960. doi: 10.1177/001316446002000104.

Rachit Dubey, Pulkit Agrawal, Deepak Pathak, Thomas L. Griffiths, and Alexei A. Efros. Investigating human priors for playing video games. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018. arXiv:1802.10217.

Sam Earle, Graham Todd, Yuchen Li, Ahmed Khalifa, Muhammad Umair Nasir, Zehua Jiang, Andrzej Banburski-Fahey, and Julian Togelius. PuzzleJAX: A benchmark for reasoning and learning. *arXiv preprint arXiv:2508.16821*, 2025. URL https://arxiv.org/abs/2508.16821.

Yanai Elazar, Akshita Bhagia, Ian Magnusson, Abhilasha Ravichander, Dustin Schwenk, Alane Suhr, Pete Walsh, Dirk Groeneveld, Luca Soldaini, Sameer Singh, Hannaneh Hajishirzi, Noah A. Smith, and Jesse Dodge. What's in my big data? In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.20707.

Shahriar Golchin and Mihai Surdeanu. Time travel in LLMs: Tracing data contamination in large language models. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.08493.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022. arXiv:2106.09685.

Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.

J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977. doi: 10.2307/2529310.

Stephen Lavelle. PuzzleScript: An open-source HTML5 puzzle game engine. https://www.puzzlescript.net/, 2013. Software and domain-specific language.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2308.03688.

Inbal Magar and Roy Schwartz. Data contamination: From memorization to exploitation. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 157–165, 2022.

NVIDIA. NVIDIA AVO reaches 100% on ARC-AGI-3: Demonstrating a frontier-level general-purpose architecture for long-horizon autonomous agents. NVIDIA Technical Blog, https://developer.nvidia.com/blog/nvidia-avo-reaches-100-on-arc-agi-3-demonstrating-a-frontier-level-general-purpose-architecture-for-long-horizon-autonomous-agents/, 2026.

Davide Paglieri, Bartłomiej Cupiał, Samuel Coward, Ulyana Piterbarg, Maciej Wołczyk, Akbir Khan, Eduardo Pignatelli, Łukasz Kuciński, Lerrel Pinto, Rob Fergus, Jakob Nicolaus Foerster, Jack Parker-Holder, and Tim Rocktäschel. BALROG: Benchmarking agentic LLM and VLM reasoning on games. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2411.13543.

Rylan Schaeffer, Brando Miranda, and Sanmi Koyejo. Are emergent abilities of large language models a mirage? In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2304.15004.

Melanie Sclar, Yejin Choi, Yulia Tsvetkov, and Alane Suhr. Quantifying language models' sensitivity to spurious features in prompt design or: How I learned to start worrying about prompt formatting. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.11324.

Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. Let me speak freely? a study on the impact of format restrictions on performance of large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, 2024. arXiv:2408.02442.

Hao Tang, Darren Key, and Kevin Ellis. WorldCoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2402.12275.

Pedro A. Tsividis, Joao Loula, Jake Burga, Nathan Foss, Andres Campero, Thomas Pouncy, Samuel J. Gershman, and Joshua B. Tenenbaum. Human-level reinforcement learning through theory-based modeling, exploration, and planning. *arXiv preprint arXiv:2107.12544*, 2021.

Archana Warrier, Dat Nguyen, Michelangelo Naim, Moksh Jain, Yichao Liang, Karen Schroeder, Cambridge Yang, Joshua B. Tenenbaum, Sebastian Vollmer, Kevin Ellis, and Zenna Tavares. Benchmarking world-model learning with environment-level queries. *arXiv preprint arXiv:2510.19788*, 2026. URL https://arxiv.org/abs/2510.19788.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.15793.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023. arXiv:2306.05685.

Sibo Zhu, Shicheng Fan, Xinyue Wang, Wenyi Wu, Kun Zhou, and Biwei Huang. RSIAgent: Autonomous exploration for recursive self-improvement in new environments. *arXiv preprint arXiv:2609.15364*, 2026. URL https://arxiv.org/abs/2609.15364.